

Factory OS — Build Log & Handover

2 September 2026 · 45 files · ~4,555 lines · 180 tests passing · core coverage 93.7%

1. Do these first — nothing works in production until they are done

Run all four from the project folder.

1. Push. The 12-function fix is committed but not pushed; Vercel keeps rebuilding the version with 13 serverless functions and failing.

```
git push
```

2. Set `AUTH_SECRET` in Vercel → Settings → Environment Variables, then **redeploy**.

Without it every endpoint answers 503 — it fails closed by design.

```
npm run auth:secret
```

3. Apply the schema to Neon. Adds the `users`, `fabricators` and `job_work` tables, the `packing_list` and `hidden` columns, and drops the old role constraint.

Safe to re-run.

```
npm run db:setup
```

4. Create the accounts. The password is typed at a hidden prompt and verified by reading it back, so "created" means "can sign in".

```
npm run user:create -- abhay --role admin
```

2. Job orders and job work — how they fit together

They are **two halves of one flow**, exactly as the Job Work Module note describes, which is why they are two screens and not two systems.

NOTE'S STEP	WHERE IT HAPPENS
1. Select fabricator (Line or External)	Create Job Order
2. Select article/style and quantity to issue	Create Job Order
3. System generates the issue slip	Create Job Order — prints the Job Card
4. When work comes back, enter received quantity	Job Orders Database
5. If payable, amount = rate × quantity	Job Orders Database

Creating a job order **is** issuing job work — it writes one `job_work` record.

The Job Card is that record's printable document, not a separate workflow.

Order corrected: the screen used to ask for the production order first and the fabricator second. The note puts the fabricator first, so it now does too — the article picker stays inert until somebody has been chosen. You decide who is free before you decide what to give them.

3. Fabricator options

Two starting options, each labelled with its type in brackets:

- **Rex Internal (Internal)** — no rate, nothing payable
- **New Durga Line (External)** — payable; rate, contact and turnaround still to be entered

They appear in the "Internal or external" dropdown on Create Job Order, and are managed under **Setup → Fabricators & lines**, where you can add more.

4. Access & accounts

The portal previously had none — anyone with the URL could read and edit every order.

Sign-in: script password hashing and HMAC-signed session cookies, both from `node:crypto`, so no new dependency. A 12-hour `HttpOnly` cookie, so the browser holds no token any script can read.

Why it cannot be forgotten: the guard lives in `wrap()`, not in each handler, so an endpoint cannot ship unprotected. A shape test asserts this across the whole `api/` tree.

Ten roles, built from your access list. A role is two things and no more: which screens it may open, and which endpoints it may change.

ROLE	CAN CHANGE	CANNOT EVEN OPEN
admin	everything	—
owner	nothing	— (sees all)
dispatch	dispatch records only	PI generation, Data & BOM, Catalogue, Parties, Stock, Plan
sales	orders, PIs, parties	Data & BOM, Catalogue, Plan, Machine load
planner	orders, PIs, dispatch, stock	Catalogue, Data & BOM, Parties
store	stock figures only	everything except Stock and Procurement
data	BOM, packing, MRP, catalogue	orders, PIs, dispatch
procurement	stock figures	BOM, orders, PIs
auditor / viewer	nothing	—

Three coarse roles could not express a dispatch clerk who records a shipment but must not raise an invoice. Run `node scripts/create-user.mjs --roles` to see all ten.

5. Documents, rebuilt from your own sheets

Packing List — a faithful copy of your 15-04-2026 sheet: 971 pairs, 49 cartons, 17 lines. Reachable from the Dispatch Book and reprintable as PDF.

The structure has two levels, and conflating them gets S.NO wrong. An S.NO is one

article/closure/colour and spans however many size rows it needs; a *carton group* is the set of sizes sharing a box, and that is what the C/N numbers follow. Your S.NO 1 holds three groups of one carton (1/49, 2/49, 3/49) while S.NO 3 holds one group of two sizes in a single box (5/49). Your whole sheet is a test fixture.

Job Card — the two-page ARMOUR 17004 form: header, size-wise cutting with total, MATERIAL ISSUED, ISSUED MATERIALS, the signature blocks, then the received/shortage/repair/rejection grids and REMARKS on page two.

Nothing is typed in from the sample. Article, fabricator, card number, date, sizes and quantities all resolve at render time, so a card for an article with no BOM prints empty rather than showing another shoe's numbers.

Job Work Challan / Internal Issue Slip — same movement, different document, chosen by the fabricator's type.

6. Bugs found and fixed — several were losing real numbers

Pairs read as cartons — 6× over-read. Your slip writes 12/18, 13/18, 1/36 — those are *pairs*. Every one was read as a carton count and multiplied by the packing rate.

One line alone came out as 1,728 against a 288-pair order. The rule now: a carton count for one size is small, so anything above ten is pairs. Each line shows which way it was read, so a wrong call is correctable rather than invisible.

One material on several component rows — 75% understated. Your revised BOM writes a

row per cut piece, so MESH 58" WHITE appears twice. The importer called that a duplicate and rejected the file; simply removing the check is worse, because the first row wins and the rest are discarded.

MATERIAL	YOUR FILE	WOULD HAVE STORED
MESH 58" WHITE	0.08	0.06
REXINE 54" WHITE	0.04	0.01

Rates now accumulate. Your header reads "Cutting componenet" — typo included — which was falling into the unrecognised-column flow, so components were silently not stored.

An order that outlived its article — blanked the whole app. One `compute()` call builds every screen, so a single orphaned order blanked the dashboard, schedule, procurement and PI list together. Orphans are now set aside and reported.

A session that invented a role. `signSession` defaulted a missing role to `"admin"`.

Any path producing a user without a role would have minted an administrator session.

A wrong password returned 500 and the lockout was dead. The failed-attempt counter

used one parameter in two incompatible type positions; real Postgres rejected the statement outright. Mocked tests all passed — only a run against a real database caught it.

Arrow keys inside a hidden password. An arrow key sends three bytes; the prompt skipped the escape and welded `[A` into the password, invisibly.

A guess wearing the reading's clothes. Match & Check was titled by the article the matcher *chose*, in bold, while the words on your slip sat beside it in grey five-point text. The ambiguity warning could also never be cleared — it survived the correction.

Cartons derived instead of counted. The dispatch screen showed `pairs ÷ rate` to two decimals — "2.67 cartons", which cannot go on a lorry.

A line totalled from the wrong rate. The PAIRS column computed `cartons × line rate`,

ignoring that SPIKE's 11X1 packs 12s/13s at 24 and size 1 at 18 — printing 1,728 where the sizes total 1,512.

One button for two intentions. Dispatch history had a single "Remove" that put pairs

back. Now: **Undo dispatch** (mis-keyed — pairs return) and **Just remove from history** (goods shipped — pairs keep counting).

Four copies of the stage order, one wrong. The stage-targets editor omitted PREPARATION and UPPER_QC and carried a PRINTING stage that isn't a stage. The production

plan read work centres in database storage order, putting PACKING second and DISPATCH third.

7. Routing & scheduling

One definition, and everything that lists stages sorts by it:

```
CUTTING → PREPARATION (printing) → STITCHING → UPPER_QC (qc & prep) → MOLDING →
```

Outside stitching adds a transit leg, not a release delay. The two transport days were being added to the *release* date, delaying cutting for a lorry that had nothing to carry yet. They now sit between STITCHING and UPPER_QC.

ORDER	CUTTING	STITCHING	TRANSIT	DISPATCH
in-house	6 Jul	8 Jul	—	12 Jul
outside	6 Jul	8 Jul	9–10 Jul	14 Jul

It books no capacity and never appears on the machine board — nothing is being made — but it is real elapsed time and every dispatch date depends on it.

8. Menu & naming

WAS	NOW	WHY
Orders & dispatch	Order Book	the factory's own name
Dispatch & packing	Dispatch Book	the factory's own name
Bulk upload (top level)	a mode inside PI generation	same job — getting orders in
Release, buried in a PI row	Create Job Order	a shop-floor decision, not a commercial record
—	Job Orders Database	receiving, shortage and payment

The slip/PI reader stays **mounted** when you switch to the spreadsheet mode — it holds an unsaved draft, and discarding a slip a clerk had already checked would be a nasty way to lose work.

9. Still needs you

WHAT	WHY IT MATTERS	WHO
15 zero-rate BOM rows (D-RING 25MM, VELCRO 20/25MM hook & loop)	The workbook loads with these flagged, but a rate of 0 orders none of a material the shoe genuinely uses.	factory
Pieces per pair, per component	Your BOM gives component <i>consumption</i> (VAMP MESH 0.06 MTR). The paper card also shows a piece count (VAMP 1824 PCS) — a different figure. The card prints consumption until you have it.	factory
New Durga Line's rate & contact	An external fabricator needs a rate per piece and a contact. Neither has been invented.	you
Real turnaround for Rex Internal	Seeded at 0 and explicitly labelled incomplete.	factory
The "Spike Blue" ambiguity	Two or more configured articles match that text. Send the article list and the tie-break can be checked.	you
Rotate the Neon password	It appeared in chat and shell history.	you
Step 6 "preparation"	Your routing listed preparation twice — after molding, before packing. Ignored for now, as agreed. If it is a real operation, name it and it can be added.	you

Two things deliberately not done. The static app shell is still served publicly — it holds no factory data, every read is an API call that returns 401, but the login page is reachable by anyone with the URL. And viewers still see write buttons on screens they can open; those return 403 with a clear reason rather than being greyed out.

10. Commands worth keeping

COMMAND	WHAT IT DOES
<code>npm run db:setup</code>	Apply the schema. Safe to re-run; prints which database it touched.
<code>npm run user:create -- <name> --role <role></code>	Create or reset an account. Verifies by reading the hash back.
<code>node scripts/create-user.mjs --roles</code>	List all ten roles with what each can do.
<code>node scripts/create-user.mjs --list</code>	Who has an account, and who is locked out.
<code>node scripts/create-user.mjs --verify <name></code>	Test a password against the stored hash — no browser involved.
<code>node scripts/create-user.mjs <name> --set-role <role></code>	Change a role without resetting the password.
<code>npm run check</code>	Full test suite, coverage thresholds and production build.

One trap worth remembering: Vercel's Hobby plan builds one serverless function per file under `api/` and allows 12. The project sits at exactly 12 — a 13th fails the *deployment* even though the build succeeds. A shape test now asserts the count. Add new endpoints as an action on an existing file, or put helpers under `api/_lib/`, which Vercel ignores.

*Every claim here was verified by running the code: the packing list against the 971-pair sheet, the BOM against the BOLT workbook, the login and lockout against a real Postgres,

and the job card rendered from live reference data.*